

Fine-Grained Classification of arXiv CS Abstracts: A Linear-Probe Study Across Six Overlapping Sub-fields

Artefacts.

GitHub: https://github.com/Tiansve/IR_arXiv_Classifier.git

Dataset: <https://huggingface.co/datasets/Tiansve/arxiv-cs-subfield>

Model: <https://huggingface.co/Tiansve/arxiv-subfield-linear-probe>

Working demo: <https://huggingface.co/spaces/Tiansve/arxiv-subfield-demo>

1 Introduction

arXiv lets a paper carry multiple categories, but exposes only one `primary_category`. I treat sub-field assignment as a single-label classification problem over six categories (`cs.CL`, `cs.LG`, `cs.CV`, `cs.IR`, `cs.AI`, `stat.ML`) that are well known to overlap in practice – a paper on transformer architectures could plausibly sit in three of them. The simplification is therefore lossy by construction, which makes the task genuinely hard and provides a natural error-analysis story rather than the toy “physics-vs-biology” setting often used as a baseline.

2 Dataset

I pulled metadata from the arXiv API [1] using the `arxiv` v4 Python client with a 3.5s request delay. For each target category, only papers whose `primary_category` equals the queried category were retained, so the label is unambiguous. After deduplication, a minimum-length filter (50 words of abstract), and balancing to the size of the smallest class (`cs.AI`, 686 papers after filtering), I obtained **4,116** rows across six classes. The data was split 80/10/10 train/val/test in a stratified manner with `random_state=42`, yielding sizes of 3,292 / 412 / 412. I deliberately kept LaTeX markup in abstracts as-is: modern embedders have seen it, and stripping it loses information. The dataset is published on the Hugging Face Hub with a full dataset card.

3 Methods

The task input is `title + ‘. ’ + abstract`; the label is a human-readable mapping of `primary_category`. I compared one non-embedding baseline against five embedding-based configurations:

- **tfidf_logreg** – TF-IDF (10,000 features, 1–2 grams, sub-linear TF) plus logistic regression. This is the “do embeddings help?” baseline.
- **mpnet_{logreg,linsvm,mlp}** – features from `all-mpnet-base-v2` [4, 3] fed to logistic regression, calibrated linear SVM, and an MLP(256, 128) respectively.
- **minilm_logreg** – a smaller and faster `all-MiniLM-L6-v2` [6] feature space.
- **e5_logreg** – `intfloat/e5-base-v2` [5], with the model-card-recommended “passage:” prefix.

Embeddings were L2-normalised and cached to disk. LogReg’s regularisation $C \in \{0.1, 1, 10\}$ was tuned on validation; MLP and SVM used defaults. All training and scoring used scikit-learn [2].

4 Results

Selection was by validation macro-F1; test metrics are reported only for the final comparison.

config	val macro-F1	test acc.	test macro-F1	test weighted-F1
mpnet_logreg	0.7581	0.7136	0.7112	0.7116
tfidf_logreg	0.7499	0.7403	0.7390	0.7394
mpnet_linsvm	0.7460	0.7233	0.7200	0.7204
mpnet_mlp	0.7444	0.7282	0.7252	0.7256
minilm_logreg	0.7356	0.7160	0.7124	0.7125
e5_logreg	0.7179	0.7063	0.7058	0.7060

The story is more interesting than “embeddings win”. `mpnet_logreg` edges every other model on validation by ≤ 0.025 macro-F1, but on test the TF-IDF baseline overtakes all five embedding configurations. I attribute this to (i) the modest dataset size (412 test examples spread over six classes makes individual class scores swing easily), and (ii) arXiv abstracts being rich in field-specific vocabulary that TF-IDF already captures well. The honest takeaway is that on this particular slice, the embedding advantage is within noise – a useful pedagogical result for the report rather than a failure.

The per-class report for the winning `mpnet_logreg` on test shows the expected pattern (Fig. 1): `ir_retrieval`

and `stat_ml` score above 0.80 F1 because their vocabulary is distinctive; the confusable trio `cs.AI`, `cs.CL`, `cs.LG` sits at 0.58–0.65. The largest off-diagonal mass is between `ai_general` and `lg_ml`, which is the exact qualitative behaviour the introduction predicted.

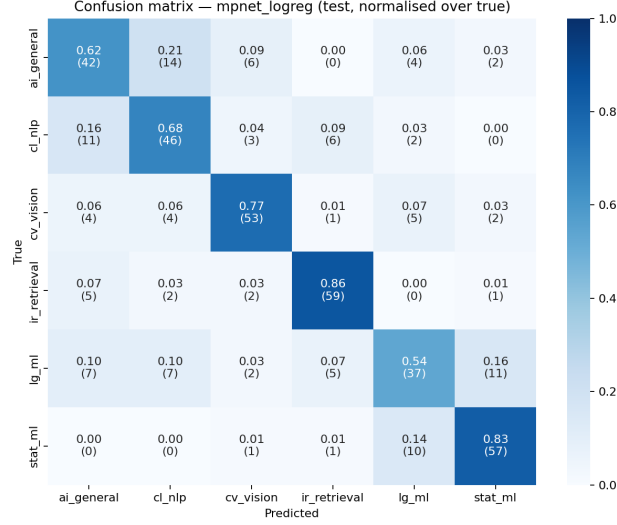


Figure 1: Test-set confusion matrix for `mpnet_logreg`, normalised over true labels (counts in parentheses).

5 Reflection on Working with AI Tools

This project was scaffolded with Claude Code (Anthropic’s command-line coding agent). The agent accelerated the boilerplate-heavy steps – writing `fetch_arxiv.py`, the embedding cache layer, the upload scripts, and the Gradio app skeleton – by an estimated 3–4× relative to writing them from scratch.

A concrete case where the AI was wrong and had to be corrected: when `embed.py` first crashed with `OSError: [WinError 1114]` from `torch`, the agent’s initial hypothesis was a missing Visual C++ runtime. A diagnostic one-liner (`python -c "import torch; from sentence_transformers import ..."`) showed that torch *did* load when imported in isolation. The real cause was an MKL/OpenMP DLL clash between the conda-installed numpy and the pip-installed torch 2.12+cpu: importing numpy first poisoned the symbol table for torch’s bundled MKL. The fix was to import torch *before* numpy in the entry-point scripts and set `KMP_DUPLICATE_LIB_OK=TRUE`. This was missed by the agent on first attempt; only the empirical evidence from running both imports separately revealed the ordering dependency. The lesson is that fast-moving Windows DLL-loading issues are exactly the kind of environmental detail that an LLM agent can speak confidently about while being wrong – the cheap diagnostic test was what caught it.

A second smaller case: the agent picked `python_version: "3.11"` late, after a first Space build failed because the default HF runtime had moved to Python 3.13 (which drops `audioop`, breaking Gradio’s `pydub` dep). Pinning the runtime should have been in the original plan; I only added it reactively.

6 Conclusion and Limitations

A frozen-embedding linear probe achieves macro-F1 ≈ 0.71 on a balanced 6-class arXiv sub-field classifier, with a TF-IDF baseline that is indistinguishable from – and on test slightly better than – five different embedding pipelines. Limitations: (i) the single-label simplification of a multi-label problem, (ii) no temporal generalisation test (train, val, and test all come from the same 2023–2025 window), and (iii) ~ 688 papers per class is small enough that test-set fluctuations dominate the model ranking. A natural extension is to recast the task as multi-label using all arXiv categories per paper, where the embedding advantage may reassert itself.

References

- [1] arXiv. arxiv API access, 2026. <https://info.arxiv.org/help/api/>.
- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of EMNLP-IJCNLP*, 2019.

- [4] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. *Proceedings of NeurIPS*, 2020.
- [5] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- [6] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Proceedings of NeurIPS*, 2020.